



Man–LLM Shared Graph Workspace: Spec-First NL-To-VIS via Serialisation

Paulis Barzdins^(✉)  and Normunds Gruzitis 

Institute of Mathematics and Computer Science, University of Latvia, Riga 1459, Latvia
paulis.barzdins@lumii.lv

Abstract. “Man–Computer Symbiosis” demands a shared workspace for cooperation, yet most modern NL-driven data systems collapse collaboration into text and execute too early on real data. We present a spec-first NL → VIS workflow that restores a shared visual workspace via serialisation: the system proposes 1–N candidate Vega-Lite “mock graphs” from the natural-language question and schema alone, LLM-populated with small plausible data for instant preview. After human edits and approval, the agreed Vega-Lite spec acts as a precise data shape contract that drives SQL generation and final rendering. The approach targets organisations like Latvia’s Ministry of Economics (EKMIN) and Central Statistical Bureau (CSP), where stakeholders ask open questions but need curated, comprehensible visuals rather than wide tables. This agreement-before-execution loop (a) prevents wide, under-structured outputs that trigger downstream transformation hell, (b) shortens iterations to seconds on mock data, and (c) preserves privacy by keeping real data local. Across replications of public NL → VIS cases and EKMIN/CSP tasks over multi-table schemas, 11/13 tasks produced acceptable visuals on the first mock and 13/13 within one round of feedback, while eliminating unnecessary wide selections. We argue Vega-Lite is a practical lingua franca for shared visual work today, and we outline design principles and limits. The contribution reframes NL → VIS evaluation from one-shot automation to cooperative productivity: the fastest path to a correct, comprehensible visualisation the user actually wants.

Keywords: NL2VIS · NL2SQL · human–LLM shared visual workspace · serialisation · Vega-Lite

1 Introduction

Licklider’s argument for “Man–Computer Symbiosis” presumes a shared workspace for people and machine cooperation, as opposed to automation [1]. In data work, that workspace should be visual when the goal is a visualisation. Yet today’s NL → SQL and NL → VIS systems mostly converse in text and execute early on real data. The result is a common anti-pattern: a direct NL → SQL path returns a wide, under-structured table that does not line up with any specific visual intent, forcing ad hoc pivots, re-aggregations, and re-runs – transformation hell – before anything communicative emerges.

We flip the order and recover a shared visual workspace via serialisation (Fig. 1). The system first proposes candidate Vega-Lite [2] “mock graphs” from the question and schema alone. Vega-Lite is a declarative visualisation grammar in a compact JSON format. A mock graph is a fully specified Vega-Lite chart, which the LLM prompt populates with small, plausible synthetic data, providing an instant, editable preview with explicit encodings, aggregations, bins, and filters. Once a candidate is approved, we treat that spec as a contract: a data shape contract enumerating the exact fields, types, groupings, filters, and sort order the chart requires. Only then do we generate SQL whose output matches that contract precisely – agreement-before-execution.

This spec-first loop delivers three practical benefits. First, it surfaces ambiguity and overcrowding cheaply, in seconds, before touching protected data. Second, it prevents wide outputs and the downstream reshaping churn that plagues execute-first workflows. Third, it supports privacy and governance constraints by keeping the LLM stage schema-only; real data are queried once, locally, under least privilege to satisfy the approved contract.

We target EKMIN/CSP settings where stakeholders ask open questions over multi-table schemas but need a handful of curated visuals, not raw tables. In EKMIN/CSP, fragmented sources and very wide tables turned every question into weeks of wrangling and re-runs – exactly the execute-first churn we replace. Technically, we sit at the junction of $NL \rightarrow SQL$ and $NL \rightarrow VIS$ but invert the usual order: propose and agree on Vega-Lite specs first, then derive SQL to fit them. Vega-Lite is used as a lingua franca: compact, declarative, widely learned by LLMs, and reliably round-trippable between human edits and machine generation.

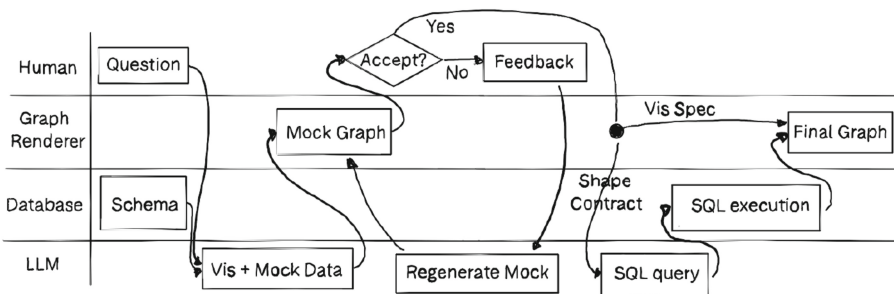


Fig. 1. System pipeline, split by roles.

Contributions:

- Spec-first $NL \rightarrow VIS$ with mock-before-execute: propose and edit candidate Vega-Lite charts from schema-only context, then lock that data shape contract and run precisely one constrained query.
- Privacy and security: ideation without data exfiltration; local execution after agreement.
- Multi-table by construction and multi-view when needed: contracts compile across joins and encourage coordinated small multiples when a single view would overload meaning.

- Evaluation lens on cooperative productivity: fastest path to an acceptable chart and avoidance of wide outputs, not brittle one-shot exactness.
- Evidence from replications and EKMIN/CSP cases: 11/13 first-mock acceptances; 13/13 within one feedback round.

2 Background and Related Work

2.1 NL → Query Over Structured Data

Natural-language interfaces to databases have matured markedly in recent years. Modern NL → SQL pipelines combine schema linking and content-aware grounding to translate free-form questions into executable SQL over multi-table schemas, often with explicit submodules for linking, generation, and debugging [3]. Parallel work extends this capability to graphs, e.g., NL → Cypher [4], framing long-term LLM memory as structured query recall. In practice, these systems are most useful to analysts who already think in relational terms and want to write queries faster; they return tables, not stakeholder-ready explanations. For non-technical users who need curated visuals, a direct NL → query path often becomes a detour: the result is a wide, under-structured table that still requires substantial shaping, cutting, and pivoting before it communicates anything.

This gap motivates integrating NL → query with visualisation intent from the start.

2.2 NL → VIS: Code-First vs Spec-First, and What Today’s Systems Assume

Execute-First Code Generation on a Single Table. a large swath of recent LLM-based NL → VIS work takes an execute-first posture: given a natural-language utterance and access to a selected table, the model emits plotting code that is immediately run on the real data, and the resulting image is inspected for correctness.

- Chat2VIS [5] demonstrates that off-the-shelf LLMs (GPT-3/Codex/ChatGPT) can directly emit Python plotting scripts from free-form NL. The prompt includes column names, types, and enumerated categorical values; raw data are not sent to the LLM, but the app must still load the data to render the chart. The workflow is one-shot with light re-asking; there is no explicit spec/mock phase, no multi-view planning, and no support for multi-table join planning. Evaluation is small-N and qualitative, and they explicitly note non-determinism and fragile styling.
- VisEval [6] codifies this dominant workflow in its benchmark harness: NL + data → Matplotlib/Seaborn code → execute → chart. Their legality/readability checks show current models are far from “fire-and-forget”: GPT-4 passes 75.27% of curated nvBench [7] cases, with quality/readability still leaving room for human judgment; adding two irrelevant tables drops pass rates by 7–20 percentage points. These results substantiate that (a) ambiguity is common enough to warrant a feasible-region notion of correctness, and (b) small changes in the surrounding schema meaningfully degrade reliability – both arguments for a mock-before-execute loop.

Spec-Templated NL → Vega-Lite Under Single-Table Scope. Other lines target a declarative spec instead of imperative code, typically on a single, pre-shaped table.

- NL4DV [8] parses NL into a JSON analytic spec and generates one or more Vega-Lite candidates. It requires raw data at initialisation to infer types and values. Ambiguity is surfaced for UI widgets, not resolved upfront; there is no explicit mock/spec approval loop and no multi-table or NL $\rightarrow$ SQL.
- ADVISor [9] blends BERT-based semantic parsing with rule-based visualisation templates to answer open questions over one table. It requires the raw table, supports a limited aggregation vocabulary and chart repertoire, and is primarily automatic and one-shot. On WikiSQL, classifier accuracies are $\sim 83\text{--}85\%$; in a user study, 72.5% of answers were rated “reasonable,” leaving $\sim 27.5\%$ needing rework – empirical support for human-in-the-loop refinement.
- ChartGPT [10] fine-tunes FLAN-T5-XL to assemble Vega-Lite from six templated sub-tasks (select/filter/aggregate; chart/encodings/sort), exposing intermediate outputs for inspection and re-generation. It assumes a single clean table and explicitly excludes joins; the prompt includes column names, types, and the top two rows, and the full CSV is required at render time. In comparative/user studies, it outperforms NL4DV and ncNet [11], yet still relies on edits and rephrasing: only 47% of successful charts were correct on the first attempt; the remainder required configuration or input changes. ChartGPT confirms that LLMs can reliably target Vega-Lite and that surfacing intermediate decisions helps – while underscoring that one-shot exactness is not the norm.

Collectively, these systems share assumptions that clash with EKMIN’s constraints:

- Single-table bias: multi-table/join scenarios are excluded or unsupported.
- Data-first execution: most require data upload or direct DB access to render; “privacy” is addressed at most by omitting raw values from prompts, not by eliminating execution on protected data during ideation.
- Execute-first iteration: ambiguity is resolved after running code on real data; there is no spec-first mock that locks intent before any heavy query.
- Limited multi-view planning: they typically emit a single chart per utterance; coordinated views/dashboards are out of scope.
- Narrow transformation coverage: wide or under-structured outputs are not a first-class concern; pivot/melt/reshape planning is largely future work.

This paper’s pipeline (Fig. 1.) addresses these gaps by proposing candidate specs with mock data from schema-only context, surfacing ambiguity early, and compiling the approved spec into SQL that returns exactly the columns and orders the visual expects.

2.3 Benchmarks and the Problem of Evaluating NL $\rightarrow$ VIS

The community has leaned on synthetic or curated datasets and exact-match metrics that fit automation-first systems but do not measure cooperative productivity.

- nvBench [7] bootstraps NL $\rightarrow$ VIS pairs from NL $\rightarrow$ SQL (Spider [12]), yielding one chart per query across seven basic types. Tables are small by enterprise standards (median < 5 columns; most 5–100 rows), no multi-view answers are included, and methods often rely on data distributions and distinct counts – precluding schema-only

workflows. Reported exact-match scores around 60–70% for neural baselines show that many cases still require redo loops.

- VisEval [6] advances beyond tree-exactness by checking legality and readability of executed code. Its “feasible region” labelling explicitly admits multiple acceptable mappings for an underspecified utterance, mirroring real practice. Yet the harness still presumes immediate execution on data and measures single-shot pass/fail – useful to quantify raw model behaviour, but orthogonal to the metrics that matter in EKMIN-style settings (time-to-acceptable-graph, iterations to agreement, and the rate at which wide, under-structured outputs are avoided).
- Surveys reinforce these tensions. Shen et al. [13] catalogue Visualisation-oriented Natural Language Interfaces and highlight that most systems operate over single tables, emphasise automatic mapping, and provide only ad hoc ambiguity resolution. They explicitly call multi-table support a future research hotspot and note the lack of standardised evaluation. Qin et al. [14] frame visualisation as inherently iterative and human-in-the-loop and elevate declarative grammars (Vega/Vega-Lite) as learnable, round-trippable specs – evidence that a serialised shared workspace is a pragmatic foundation today.

These benchmarks and surveys point to two conclusions. First, ambiguity and underspecification are endemic; feasible regions, top-k candidates, and human edits are the rule, not the exception. Second, existing metrics undercount what matters for stakeholder-facing analytics with expensive backends: preventing avoidable executions, minimising re-shaping churn, and converging to agreed visuals quickly.

2.4 Shared Workspaces, Serialisation, and the Limits of Vision-Only Tooling

Licklider’s “Man–Computer Symbiosis” from 1960 [1] argues that progress in the long interim before full automation depends on a shared workspace where humans and machines co-operate rather than one replacing the other. Contemporary multimodal LLM demos that turn sketches into UIs (e.g., MakeReal [15], Uizard [16]) hint at that future but remain imprecise marketing tactics for visualisation: they lack explicit semantics, are brittle under small changes, and provide no guaranteed round-trip between what the human means and what the model executes.

Textual serialisation fills that gap now. Declarative specs like Vega-Lite, together with simple tabular encodings (TSV/CSV), act as a lingua franca that both humans and LLMs can co-edit deterministically and render identically. Adjacent ecosystems (Mermaid, PlantUML, Graphviz) show that text-first diagramming supports reliable round-trips; the visualisation literature [14] and systems like NL4DV and ChartGPT confirm Vega-Lite is both expressive and model-learnable. This paper’s workflow operationalises Licklider’s premise for data work: use serialised specs as the shared desk surface; mock with small plausible data to resolve ambiguity cheaply; the agreed spec then acts as the data shape contract that drives SQL and final rendering.

2.5 Positioning and Contributions Relative to Prior Work

Relative to NL → query and NL → VIS systems above, this paper contributes:

- Spec-first, mock-before-execute. Unlike execute-first NL $\rightarrow$ VIS (Chat2VIS; VisEval’s default pipeline), ambiguity is surfaced and resolved on mock data before any heavy query runs, cutting iteration costs when backends are slow or data are sensitive.
- Schema-only ideation with a real privacy story. Where most NL $\rightarrow$ VIS methods require data upload or DB execution during ideation (NL4DV, ADVISor, ChartGPT), this paper’s loop keeps sensitive data local until a spec is approved; only then is SQL generated and run.
- Multi-table by construction. Rather than excluding joins (ChartGPT) or assuming a single table (NL4DV, ADVISor, Chat2VIS), this paper’s data shape contracts provide a target that the NL $\rightarrow$ SQL stage compiles to across multi-table schemas – aligning with the realities flagged as a “hotspot” by Shen et al. [13] and operationalised by modern NL $\rightarrow$ SQL pipelines [3].
- Multi-view planning up front. Single-chart biases in current NL $\rightarrow$ VIS are replaced with coordinated multi-view proposals at the mock stage when one chart would overload meaning.
- A reframed evaluation lens. Building on evidence from nvBench and VisEval that single-shot exactness is brittle, and ambiguity is pervasive, this paper’s evaluation targets cooperative productivity: time-to-acceptable-graph, iterations to agreement, and avoidance of wide, under-structured outputs.

In short, prior work shows that LLMs can emit plotting code or Vega-Lite specs from NL when given a single, clean table – and that humans routinely need to intervene. This paper’s contribution is to make that intervention the centrepiece: a serialised shared workspace that reaches agreement before any data leaves protected systems or long queries are run, and that treats the approved spec as a contract the database must satisfy rather than as an after-the-fact interpretation of whatever a query happened to return.

3 Requirements and Design Goals

We target stakeholder-facing analytics over relational data, where open NL questions should yield a small number of clear charts rather than wide tables. From the EKMIN/CSP context and preliminary trials, we derive the following design goals.

Cooperation over Automation. Keep the human in the loop. Front-load human/LLM alignment so the first costly step (data extraction/wrangling) happens only after intent is concrete.

Curation over Mass Production. Visualisations are for humans. The goal is communicative, comprehensible visuals, not chart spam. Prefer a small set of deliberate, edited charts that answer the question clearly, instead of mass production.

Shared Workspace via Serialisation. Licklider’s requirement for man-machine cooperation. To roughly see and agree on output quickly. Available today through serialisation. Vega-Lite and tabular encodings (TSV) as a common workspace with immediate visual rendering, that both human and LLM can co-edit.

Data Shape Contracts. Mock visualisation before data query. Once mock visualisation is human approved, that Vega-Lite spec acts as data shape contract, so SQL query output matches the visualisation input requirement exactly. To completely avoid over-wide output tables that require unmanageable transformations on visualisation side.

Multi-View Planning up front. A single question is not always best served by a single visualisation. At the mock stage, the system or the user can propose and pivot to 1 to N coordinated charts, to avoid an overcrowded and confusing single output and to surface complementary perspectives early.

Minimal, Reproducible Stack. Use stable, text-based components that others can reproduce on their own schemas without heavy tooling, and that will age well as models improve:

- Vega-Lite for declarative chart specs.
- TSV/CSV for small data payloads and mock data.
- SQL for retrieval; jq for TSV → JSON quoting when embedding data into specs.
- Prompt templates for the two-step loop (spec-first, then SQL).

This avoids a brittle monolith that rots with dependencies. The pipeline runs in any environment that can render Vega-Lite (browser/notebook/CLI) and execute SQL locally. If Vega-Lite is superseded, the same pattern transfers to a successor grammar with minimal change. The emphasis is on transparency, portability, and longevity: the workflow should still be repeatable a decade from now, only better as LLMs improve.

Security and Privacy by Design. Treat the LLM and orchestration layer as untrusted with respect to production data; keep sensitive data local until after agreement:

- Schema-only ideation. During the mock/spec stage, the LLM sees the schema and comments but never raw rows. Mock data are synthetic.
- Local/controlled execution. Execute generated SQL in a controlled environment; paste or pipe only the minimal result (matching the contract) into the renderer. An offline/air-gapped path is supported.
- Least-privilege queries. The contract constrains SELECTs to only the required columns, aggregations, filters, and sort order; no wide SELECT * or unnecessary joins.
- No data exfil during iteration. Agreement-before-execution prevents “probe” queries on protected data just to see what’s there.
- Auditability. Persist all artifacts – question, approved Vega-Lite spec, data shape contract, generated SQL, and checksums of outputs – for compliance and reproducibility.
- Optional local models. Where required, swap remote LLM calls for on-prem/local models without changing the workflow.

These measures match EKMIN/CSP constraints and generalise to other settings with strict data-governance or Data Loss Prevention (DLP) requirements.

4 Concept and Workflow

4.1 Mock Stage – Preview and Agree

Given a natural-language question and a relational schema (with optional comments), the system proposes one or more candidate charts as Vega-Lite specifications populated with small, plausible mock data. This establishes a serialised shared workspace the human can see and edit immediately.

- Inputs – NL question, schema, optional schema comments/quirks.
- System proposal – 1-N Vega-Lite specs with synthetic mock data sized for instant preview.
- Human-in-the-loop refinement – Accept, edit, split, or merge candidates; Iterate until the intended mappings and scope are unambiguous.
- Agreement and shape lock – The approved spec becomes the single source of truth. It is the data shape contract: the exact output fields, types, groupings/aggregations, filters, sort orders, and any required derived fields per chart/view.

This mock-before-execute loop keeps iteration times to seconds, surfaces under-specification early, and naturally supports multi-view answers when one chart would overload meaning.

4.2 Output Stage – Contract, Query, and Render

Once the spec is locked, execution becomes a constrained translation and rendering problem.

- Contract-driven SQL – The system treats the approved spec as a data shape contract and generates SQL whose output matches that contract exactly – no wide SELECT, no extra columns.
- Local, least-privilege execution – SQL is run in a controlled environment. Only the minimal result needed by the spec is exported (e.g., TSV/CSV).
- Render and finalise – The result is embedded or linked into the approved Vega-Lite spec and rendered.

4.3 Failure Modes and Recovery

The workflow is designed to surface ambiguity and risk early, before expensive execution. We distinguish conceptual catches (Sect. 4) from technical defenses (Sect. 5).

- Underspecified intent (e.g., “by age” without bins or units)
 - Caught at: mock stage. Candidates must choose explicit binning/aggregation, making the choice visible.
 - Human action: confirm or adjust bin edges, aggregations, and encodings.
- Overcrowded single-view answers
 - Caught at: mock stage when marks or legends overwhelm the view.
 - Human action: split into coordinated small multiples or facet; remove non-essential encodings.
- Wrong field or proxy choice
 - Caught at: mock stage via selected fields and derived expressions.

- Human action: redirect to the appropriate column or accept a documented proxy.
- Hidden schema quirks (inconsistent enums, sentinel values, null patterns)
 - Caught at: mock stage if called out in schema comments; otherwise during first real render if distributions look off.
 - Human action: add/adjust filters or mappings in the spec; update schema comments for future runs.
- Contract/query mismatch
 - Caught at: compile or render time if returned columns/types do not match the contract.
 - Human action: adjust spec or regenerate SQL under the unchanged contract.
- Governance or privacy violations (accidental over-selection)
 - Caught at: compile time by the least-privilege contract.
 - Human action: narrow scope or redact sensitive fields before approval.

5 System and Implementation

The prototype is operator-mediated: system is executed with a clean chat API, an online Vega-Lite renderer, and per-task SQL runners.

5.1 Prompt One – Query to Mock Graph

Responsibilities: read schema, map the question to fields/derivations, propose 1–N Vega-Lite specs, include small plausible mock data.

Locked prompt (for evaluation case studies):

Instructions:

Below is a query and a SQL schema. Output Vega-lite code fulfilling the query, inputting fake mock-data for now. Once the visualisation is agreed upon, you will generate an SQL query matching the Vega-lite visualisation exactly.

Query:

```
{query}
```

Schema:

```
{schema}
```

Though the prompt was locked for the evaluation case studies, in practical use there are potential additions to achieve more tailored results:

To Avoid Full Table Data in Vega-Lite. The locked prompt allows importing full data to Vega-Lite and processing it there. If this is unwanted behaviour (say for CSP with millions of data entries) the prompt can be extended thus:

```
{Instructions ...}
```

```
mock-data is of custom sql query output, not full data
table
```

```
{Query and Schema}
```

To Specify Data Output. For the small evaluation case studies this wasn't important, but for larger runs you might want to specify the SQL query output format, to lock in a method of importing it to Vega-Lite. For EKMIN tasks I specified TSV output, then the Vega-Lite spec is extended with TSV data specifications, and data import runs without surprises.

To Supplement Poor SQL Schemas. The initial prompt can contain comments on specific schema quirks. This can also be further mediated in the back and forth before spec approval. Reasonable schemas (all evaluation case studies) need no special treatment. CSP schema has a few quirks with data coverage issues that need special treatment.

5.2 Prompt Two – Lock Data Shape and Generate SQL Query

Prompt one initiates a chat loop, which finishes upon user acceptance of the Vega-Lite spec, where acceptance is the sending of prompt two (also locked for evaluation):

```
perfect! Now output an SQL query matching the schema,
that returns data in a way that exactly matches the shape
used for the Vega-Lite visualisation.
```

This outputs the SQL query and marks the privacy line, behind which LLMs are no longer used and everything can be run locally and securely.

5.3 Query Execution and Final Rendering

Run the SQL query.

Place SQL output into Vega-Lite. For evaluation case studies this process was manual, differing from case to case. For EKMIN use this was pre-agreed to be done via `tsv`, with the following command making the file placeable into the Vega-Lite spec (e.g. making white spaces explicit as text):

```
jq -Rs '.' out.tsv
```

Vega-Lite renderer will update upon receiving the real data, and the final visualisation is rendered.

6 Evaluation Design

Goal and Rationale. We evaluate the spec-first, mock-before-execute workflow on what matters for stakeholder-facing analytics: how quickly and reliably a user reaches an acceptable visual answer with zero unnecessary executions and no wide, under-structured outputs. Prior work shows why automation-centric exact-match metrics are insufficient for this setting: nvBench synthesises single-chart pairs from NL $\rightarrow$ SQL over small tables; VisEval still assumes execute-first code and scores one-shot legality/readability in a “feasible region,” explicitly acknowledging multiple acceptable mappings. Our evaluation therefore centres on cooperative productivity rather than single-shot exactness.

Evaluation Questions

- EQ1: Does a spec-first mock stage reduce avoidable executions and post-query reshaping relative to execute-first NL $\rightarrow$ VIS?
- EQ2: How many iterations are typically required to reach an acceptable chart, and how long does it take?
- EQ3: Does the contract-driven handoff eliminate wide outputs and redundant columns in practice?
- EQ4: Can the approach scale to complex real-world multi-table schemas and motivate multi-view answers when a single chart would overload meaning?

Task Suites. We adopt case-study replication as in Chat2VIS and ADVISor, aiming for comparative parity while measuring the cooperative productivity of a spec-first workflow.

- Chat2VIS replication (all case studies)
 - Scope: We replicate every case study reported in Chat2VIS, using the same data sources (e.g., nvBench-sourced tables; public CSVs referenced in their paper/examples).
 - Purpose: Provide an apples-to-apples contrast with an execute-first, one-shot code-generation workflow over single tables.
 - Expectation from related work: Chat2VIS acknowledges non-determinism, fragile styling, and a lack of explicit mock/spec phases; we test whether early agreement shortens the path to acceptable visuals.
- ADVISor replication (representative case studies until saturation)
 - Scope: We reproduce the canonical ADVISor case families on their public datasets (e.g., energy production), then stop once additional cases become monotone (same semantic pattern, same single-table mapping).
 - Purpose: Compare against a template-driven, one-shot NL $\rightarrow$ Vega-Lite system that assumes a single pre-shaped table.
 - Expectation from related work: ADVISor’s own user study found that 27.5% of results needed rework; we test whether a mock stage absorbs that rework upfront and whether multi-view proposals emerge when a single view overloads meaning.

- EKMIN real-world cases (complex and large multi-table; governance-constrained)
- Scope: Questions sourced from Latvia’s Ministry of Economics over ~40 tables in the CSP-aligned schema. We select a real batch of tasks that were done and delivered manually before the development of this system.
- Purpose: Demonstrate the intended target: big schemas (with quirks), large tables, zero data exfil during ideation, and willingness to propose multiple coordinated views when appropriate.

Protocol

- Prompts: Use the frozen prompts from Sect. 5 for all case studies. No prompt engineering mid-run.
- Model settings: Temperature 0.1; one fixed model (OpenAI gpt-5 with reasoning).

Threats to Validity and Mitigations.

- Small-N and selection bias: We replicate all Chat2VIS cases and a saturated set from ADVISor to limit cherry-picking; EKMIN tasks are pre-listed before running.
- Single-operator bias: We freeze prompts, show all iterations.
- Non-determinism: We report one canonical run per task.

7 Results

Across 13 tasks (Fig. 2.) (replications of Chat2VIS and ADVISor cases plus EKMIN/CSP tasks over complex multi-table schemas), 11/13 (85%) were acceptable on the first mock; all 13/13 (100%) reached an acceptable visual within a single round of feedback. In 8/13 cases, the final visual was judged clearer than “gold” (“gold” is published best visualisation from reference paper), due to clearer visuals, more appropriate tick marks, extra analytics. No run emitted a wide or under-structured output: every SQL was generated from the approved spec’s data shape contract and executed exactly once per task (Table 1.). Minor corrections surfaced without changing contracts (e.g., x-axis year tick formatting; preferred stack order in one EKMIN area chart).

Table 1. Results summary.

Metric	Result
Tasks (N)	13
First-mock acceptable	11/13 (85%)
Completed within 1 feedback	13/13 (100%)
“Improved over gold”	8/13
Wide outputs emitted	0/13
SQL executions per task	1

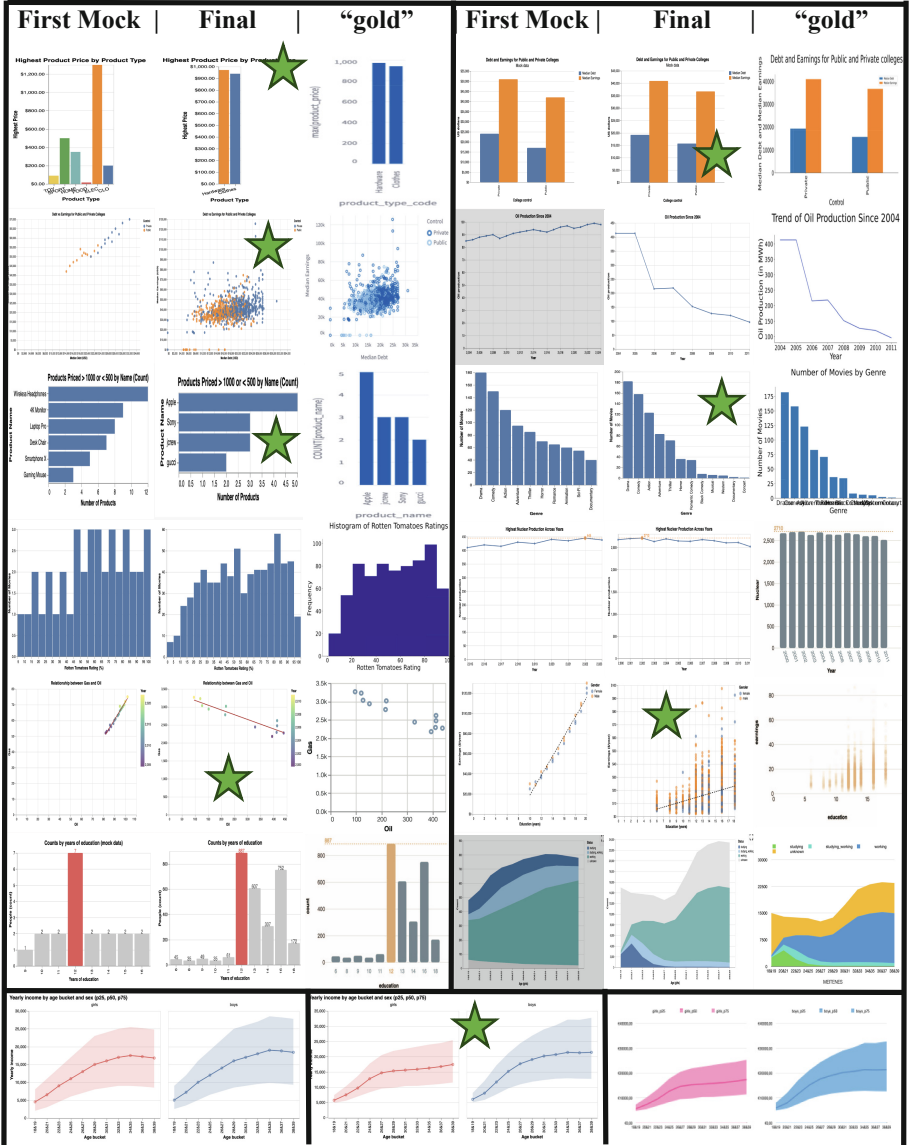


Fig. 2. First mock, final, and “gold” (published best visualisation from reference papers) for each of 13 tasks. Greyed out = accepted after one feedback; the rest = accepted at first mock; star marks cases judged clearer than the published gold.

8 Discussion

This work uses an appropriate graphing language (Vega-Lite over python plotting libraries), current best LLM models, and leaves human in the loop for cheap and rapid tweaks if needed. In practice, this explains most of the observed gains. We observed an

85% first-mock acceptance rate. The most recent and capable NL → VIS models report at most 75% perfect scores over whole evaluation datasets. Our high score is likely due to the use of Vega-Lite over python plotting libraries, and due to using newer LLM models. Vega-Lite takes some of the formatting responsibility off the system, making it easier to pass on first go. LLM improvements likely also gain some percentage points.

What is more, those 15% could potentially be avoidable if that were the goal, allowing our score to potentially push even upwards of 85% (though this is not on a full evaluation set). One needed feedback was for improper year tick mark formatting (“2,018” instead of “2018”) – likely easily catchable if prompt included “make sure tick marks are formatted accordingly to...”. The other wasn’t even a mistake, just feedback on user preference – that the stacked areas were visually preferred to be inverted. Thus, if automation best score was a goal, this approach can likely place you among current top performers even without a human-in-the-loop.

Eight of the system results were deemed improvements over the “gold” standard. Part of these were expected, as the “gold” was simply the best performer of a previous paper, which doesn’t always mean good. But some were a genuine surprise, using specialist graph types, or stacked visualisations, without any hiccups.

This isn’t to suggest that the graphs on first go are so good, that the human in the loop is useless. There were still signs of non-determinism (e.g. angled tick labels with no clear reason, almost obstructing one another) and some of the bolder system choices would need to be approved (splitting male and female income to two graphs, instead of plotting together).

With the human present though, this has shown to be a reliable system, both for simple plotting as in the evaluation case studies, as well as for more serious tasks over complex real-world multi-table schemas, as can be seen in the highlight in Fig. 3. In these EKMIN/CSP examples, the system arguably outperformed the human-crafted plots.

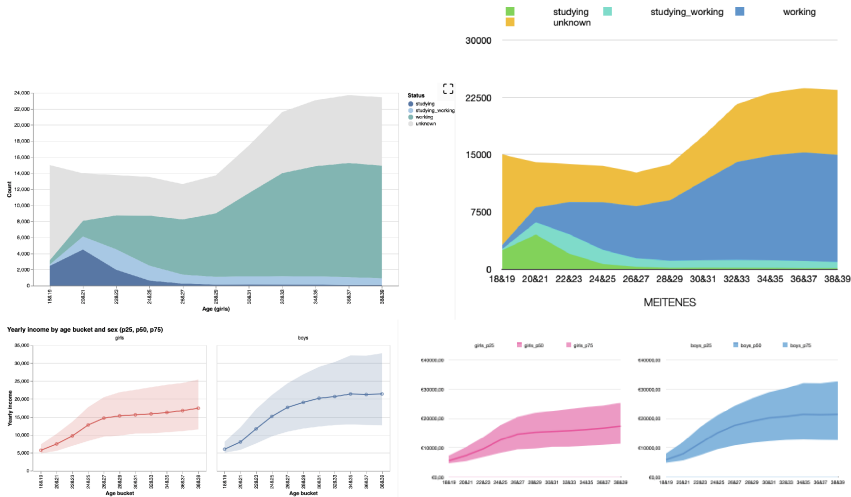


Fig. 3. System outputs (left) vs manual human-crafted (right) on real EKMIN/CSP use-cases.

9 Conclusion

We presented a spec-first, LLM-assisted NL $\rightarrow$ VIS workflow that restores a shared visual workspace via serialisation. The system proposes mock Vega-Lite charts from the question and schema alone, then treats the approved spec as a data shape contract that drives least-privilege SQL and final rendering. This agreement-before-execution reframes success from brittle one-shot exactness to cooperative productivity: the fastest path to a correct, comprehensible visual that the user actually wants. In replications and EKMIN/CSP cases, 11/13 tasks were acceptable on the first mock and 13/13 within one round, while eliminating wide, under-structured outputs.

Vega-Lite serves as a practical lingua franca today: compact, declarative, and reliably round-trippable between human edits and model generation. The approach aligns with privacy and governance constraints by keeping real data local until after agreement and by executing a single constrained query.

As multi-modal LLMs develop, their visual acumen might eventually replace the need for serialisation based shared workspaces, but as Licklider said “the {interim years} may be 10 or 500, but those years should be intellectually the most creative and exciting in the history of mankind” [1].

Acknowledgments. This study was funded by the EU Recovery and Resilience Facility’s project “Language Technology Initiative” (2.3.1.1.i.0/1/22/I/CFLA/002).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Licklider, J.C.R.: Man-computer symbiosis. *IRE Trans. Hum. Fact. Electron. HFE*-**1**, 4–11 (1960)
2. Satyanarayan, A., Moritz, D., Wongsuphasawat, K., Heer, J.: Vega-Lite: a grammar of interactive graphics. *IEEE Trans. Visual Comput. Graphics* **23**(1), 341–350 (2016)
3. Luo, Y., Li, G., Fan, J., Chai, C., Tang, N.: Natural language to SQL: state of the art and open problems. *Proc. VLDB Endowment* **18**(12), 5466–5471 (2025)
4. Barzdins, P., Gruzitis, N.: OB-GRAG: LLM-assisted graph creation and querying via domain-specific ontology (2025)
5. Maddigan, P., Susnjak, T.: Chat2Vis: generating data visualizations via natural language using ChatGPT, codex and GPT-3 large language models. *IEEE Access* **11**, 45181–45193 (2023)
6. Chen, N., Zhang, Y., Xu, J., Ren, K., Yang, Y.: VisEval: a benchmark for data visualization in the era of large language models. *IEEE Trans. Visual Comput. Graph.* **31**(1), 1301–1311 (2024)
7. Luo, Y., Tang, N., Li, G., Chai, C., Li, W., Qin, X.: Synthesizing natural language to visualization (NL2VIS) benchmarks from NL2SQL benchmarks. In: *Proceedings of the 2021 International Conference on Management of Data*, pp. 1235–1247 (2021)
8. Narechania, A., Srinivasan, A., Stasko, J.: NL4DV: a toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Trans. Vis. Comput. Graph.* **27**(2), 369–379 (2020)

9. Liu, C., Han, Y., Jiang, R., Yuan, X.: Advisor: automatic visualization answer for natural-language question on tabular data. In: 2021 IEEE 14th Pacific Visualization Symposium (PacificVis), pp. 11–20 IEEE (2021)
10. Tian, Y., et al.: ChartGPT: leveraging LLMs to generate charts from abstract natural language. *IEEE Trans. Vis. Comput. Graph.* **31**(3), 1731–1745 (2024)
11. Luo, Y., Tang, N., Li, G., Tang, J., Chai, C., Qin, X.: Natural language to visualization by neural machine translation. *IEEE Trans. Vis. Comput. Graph.* **28**(1), 217–226 (2021)
12. Yu, T., et al.: Spider: a large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 3911–3921 (2018)
13. Shen, L., et al.: Towards natural language interfaces for data visualization: a survey. *IEEE Trans. Vis. Comput. Graph.* **29**(6), 3121–3144 (2022)
14. Qin, X., Luo, Y., Tang, N., Li, G.: Making data visualization more efficient and effective: a survey. *VLDB J.* **29**(1), 93–117 (2020)
15. Make real homepage. <https://makereal.tldraw.com>. Accessed 14 Mar 2026
16. Uizard wireframing page. <https://uizard.io/wireframing/>. Accessed 14 Mar 2026